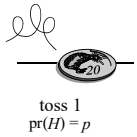
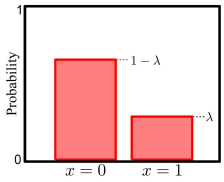
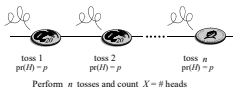
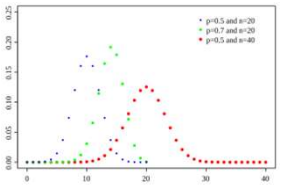
Agenda

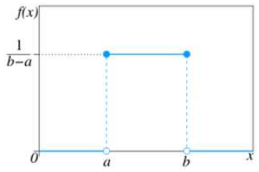




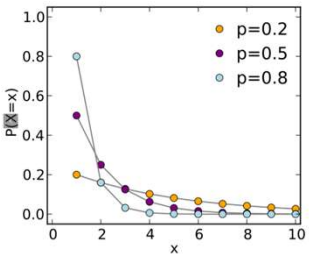
- Last time
  - Some distributions used over and over again
  - Hypergeometric
  - Continuous distribution
- Today
  - PRNG
  - Uniform distribution
  - Random subset











2

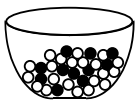
Hypergeometric

$$P(X = i) = \frac{\binom{N}{i}\binom{M}{n-i}}{\binom{N+M}{n}}$$

- If we sample *n* balls at random from a finite population without replacement, and count X, the number of black balls
- Consider X_i which has value 1 if the ith trial produces a black ball and 0 otherwise
  - Let $T = N + M$
  - Clearly, $P(X_1 = 1) = \frac{N}{T}$
  - Assume $P(X_k = 1) = \frac{N}{T}$
- Although the X_i's are **not independent**, every position in the sequence has the same marginal probability

$$\begin{aligned} P(X_{k+1} = 1) &= P(X_{k+1} = 1 \mid X_k = 1)P(X_k = 1) \\ &\quad + P(X_{k+1} = 1 \mid X_k = 0)P(X_k = 0) \\ &= \frac{N-1}{T-1}\frac{N}{T} + \frac{N}{T-1}\frac{M}{T} \\ &= \frac{N(N-1)+NM}{T(T-1)} \\ &= \frac{N(T-1)}{T(T-1)} \\ &= \frac{N}{T}. \end{aligned}$$

N black balls
M white balls
Choose n balls
How many black?

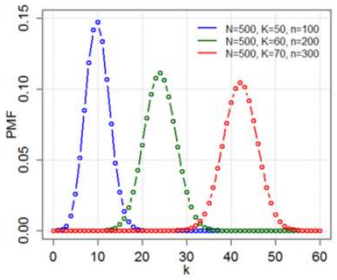


3

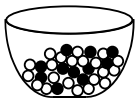
Hypergeometric

$$P(X = i) = \frac{\binom{N}{i}\binom{M}{n-i}}{\binom{N+M}{n}}$$

- If we sample n balls at random from a finite population and count X, the number of black balls
- How do we choose "at random" 100 people from a group of 200?



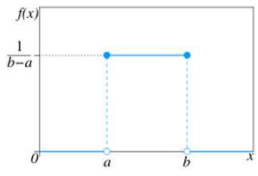
N black balls
M white balls
Choose n balls
How many black?



4

Uniform distribution

- Uniform random variables, especially over the [0,1] interval are very important, in developing programs to draw samples from other distributions
- That's the theory
  - In practice, we assume that we have access to a generator that can generate a canonical random variable
    - The random number generator is actually a pseudo-random number generated



119

5

Pseudorandom Numbers

- Concept of a seed
- For any initial seed, subsequent numbers appear to be random, cannot be predicted …

This is desirable

6

Demo

7

Random Numbers: Produce integers

- rand() is often used

What's Wrong With This Code?

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main() {
    srand(time(NULL));
    for (int i = 0; i < 16; ++i) {
        printf("%d ", rand() % 100);
    }
    printf("\n");
}
```

Adapted from rand() considered harmful (Stephan T. Lavavej)

9

Random Numbers (API)

What's Right With This Code?

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main() {
    srand(time(NULL));
    for (int i = 0; i < 16; ++i) {
        printf("%d ", rand() % 100);
    }
    printf("\n");
}
```

All required headers are included!

All included headers are required!

Headers are sorted!

One True Brace Style!

%d is correct for int!

10

Random Numbers (API)

- rand() is often used

What's Wrong With This Code?

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main() {
    srand(time(NULL));
    for (int i = 0; i < 16; ++i) {
        printf("%d ", rand() % 100);
    }
    printf("\n");
}
```

Frequency: 1 Hz!

warning C4244: 'argument' : conversion from 'time_t' to 'unsigned int', possible loss of data

32-bit seed!

Range: [0, 32767]

Non-uniform distribution!

Linear congruential → low quality!

11

Random Numbers (API)

- Assume rand() is uniform in [0, 32767]

```
//      [0, 99]  src → [0, 99] dest
//      [100, 199]  src → [0, 99] dest

//      …

//      [32700, 32767]  src → [0, 67] dest
```

- The output is not uniform

12

Hello random world

- Mersenne twister
  - Not cryptographically secure
- Automatic seed

```python
import random
//random.seed(42)
n = int(input())
a = []
for i in range(n):
    a.append(random.randint(1, 100))
for i in range(n - 1):
    min_index = i
    for j in range(i + 1, n):
        if a[j] < a[min_index]:
            min_index = j
    a[i], a[min_index] = a[min_index], a[i]
for x in a:
    print(x, end=" ")
print()
```

13

Hello, "Random" World!

```cpp
#include <iostream>
#include <random>
int main() {
    std::mt19937 mt(1729);
    std::uniform_int_distribution<int> dist(0, 99);
    for (int i = 0; i < 16; ++i) {
        std::cout << dist(mt) << " ";
    }
    std::cout << std::endl;
}
```

Engine: [0, 2^{32})

Deterministic 32-bit seed

Distribution: [0, 99]

Note: [inclusive, inclusive]

- Engine: fast, seedable, reproducible
- Obsessive implementors (perfect uniformity), deterministic but may vary across platforms

14

Pseudorandom Numbers

This is desirable

- Concept of a seed
- For any initial seed, subsequent numbers appear to be random, cannot be predicted …
- … until we reach the period after which the numbers reappear in the same order so
  - We need a long period
  - Need efficient pseudo random numbers generator (prng) (should be as cheap as anything)

15

Why uniform?

- Uniform random variables, especially over the [0,1] interval are very important, in developing programs to draw samples from other distributions
- *We assume the existence of a canonical random number generator*

119

16

Generating a random subset

- Choosing a population for a medical trial
- Generate a random subset B_k of size k from a set $A = a_1, a_2, \ldots, a_n$ of size n
  - All the $\binom{n}{k}$ subsets must be equally likely
- Consider another problem
  - Generate a discrete random variable X having the pmf

$$P\{X = x_j\} = p_j, \quad j = 0, 1, \cdots, \sum_j p_j = 1$$

122

17

Generating a discrete pmf

- Generate a discrete random variable X having the pmf
- Example: 0.2, .15, .25, 0.4
- Algorithm
  - Generate a sequence of random numbers U that is uniformly distributed over [0, 1]

$$P\{X = x_j\} = p_j, \quad j = 0, 1, \cdots, \sum_j p_j = 1$$

$$X = \begin{cases} x_0 & \text{If } U < p_0 \\ x_1 & \text{If } p_0 \le U < p_0 + p_1 \\ \vdots & \vdots \\ x_j & \text{If } \sum_{i=0}^{j-1} p_i \le U < \sum_{i=0}^{j} p_i \\ \vdots & \end{cases}$$

$$0 < a < b < 1 \longrightarrow P\{a \le U < b\} = b - a$$

$$P\{X = x_j\} = P\left\{\sum_{i=0}^{j-1} p_i \le U < \sum_{i=0}^{j} p_i\right\} = p_j$$

122

18

Generating a random subset

- Generate a random subset B_k of size k from a set $A = a_1, a_2, \ldots, a_n$ of size n
  - All the $\binom{n}{k}$ subsets must be equally likely
- Define indicator variable

$$I_j = \begin{cases} \mathbf{1} & \{a_j \in B_k\} \\ \text{else} & 0 \end{cases}$$

122

19

Generating a random subset

- $P(I_1 = 1) = k/n$
  - Denominator all outcomes: $\binom{n}{k}$ ways to pick k objects out of n
  - Numerator (favourable outcomes)
    - Item 1 is picked, there is only one way of picking it, and
    - we have to pick k-1 items out of the remaining n-1 items i.e. $\binom{(n-1)}{(k-1)}$
  - Probability: favourable divided by total.
- If I_1 is 1, then having chosen I_1, the remaining k-1 items must be chosen to be "equally probable"
  - $P(I_2 = 1 \mid I_1 = 1) = \frac{k-1}{n-1}$

123

20

Generating a random subset

- $P(I_1 = 1) = k/n$
  - $P(I_2 = 1 \mid I_1 = 1) = \frac{k-1}{n-1}$
  - If I_1 is 0, $P(I_2 = 1 \mid I_1 = 0) = \frac{k}{(n-1)}$
- Therefore, $P(I_2 = 1) = \frac{(k-I_1)}{(n-1)}$
- Continuing,

← That could be written as $P(I_2 = 1 \mid I_1)$

$$P(I_j = 1 \mid I_1, \ldots, I_{j-1}) = \frac{k - \sum_{i=1}^{j-1} I_i}{n - (j-1)}$$

123

21

Generating a random subset

- Use an algorithm similar to the earlier algorithm
  - Generate at most n random numbers with the stated probabilities
  - Choose item 1 if U_1 < k/n else skip
  - Choose item 2 if U_2 < (k-1)/(n-1) else skip
  - …

$$U_j \sim \text{Uniform}(0, 1)$$

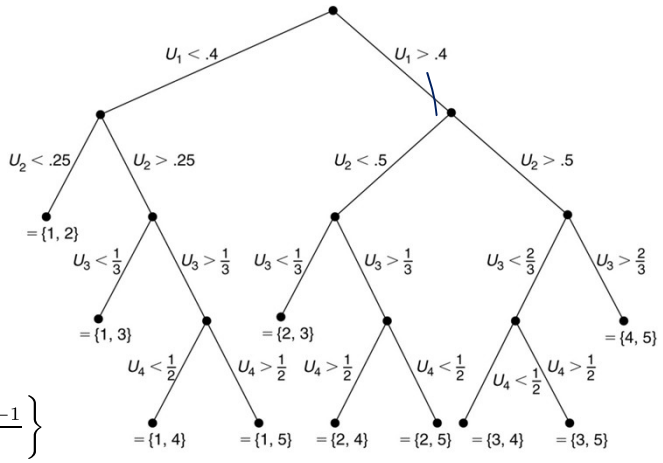
$$I_j = \mathbf{1}\left\{U_j < \frac{k - I_1 - \cdots - I_{j-1}}{n - j + 1}\right\}$$

125

22

Example

k=2, n=5



$$I_j = \mathbf{1}\left\{U_j < \frac{k - I_1 - \cdots - I_{j-1}}{n - j + 1}\right\}$$

23

Generating a random subset

- Algorithm stops
  - Step j if $I_1 + I_2 + \cdots + I_j = k$
    - Random subset B_k contains those indices whose indicator values are 1
  - Or, if the number of unfilled entries in the random subset B_k equals the number of remaining elements in A

$$U_j \sim \text{Uniform}(0, 1)$$

$$I_j = \mathbf{1}\left\{U_j < \frac{k - I_1 - \cdots - I_{j-1}}{n - j + 1}\right\}$$

125

24